PROYECTO DE

PROGRAMACIÓN II

INTEGRANTES

Nombre Completo	Carné	Usuario GitHub
Rubí Ester Marroquín Carias	0905-24-11744	rmarroquinc9
Ericson José Cristales Rivera	5190-24-10720	Littl3Shark
Kevin Ariel Navas Arias	5990-22-11393	knavasa2-gif

CATEDRATICO

Ing. Elder Amilcar Herrera Cifuentes

ÍNDICE GENERAL

INTRODUCCIÓN 1
1.1 Justificación 1
2. OBJETIVOS 2
2.1 Objetivo General 2
2.2 Objetivos Específicos 2
3. MARCO TEÓRICO 3
3.1 Estructuras de Datos Implementadas 3
3.2 Patrón de Diseño: MVC y SOLID 3
4. METODOLOGÍA Y DISTRIBUCIÓN DE RESPONSABILIDADES 4
5. DESARROLLO DEL PROYECTO 5
5.1 Estructura del Repositorio 5
6. MANUAL DE USUARIO 5
6.1 Requisitos Previos 5
6.2 Instalación y Ejecución 5
6.3 Flujo de Uso 6
7. EVIDENCIAS DE DESARROLLO Y REFACTORIZACIÓN 6
7.1 Capturas de Pantalla del Sistema 6
8. EVIDENCIAS DE TRABAJO EN EQUIPO 8
8.1 Contribuciones en GitHub 8
8.2 Sesiones de Trabajo 8
9. CONCLUSIONES 9
10. RECOMENDACIONES 10
11. BIBLIOGRAFÍA 11
12. ANEXOS 12

INTRODUCCIÓN

El presente documento describe el desarrollo del proyecto "Sistema de Gestión de Turnos", una aplicación de consola desarrollada en Java 21 que administra eficientemente la atención de clientes en una oficina de servicios.

El sistema implementa estructuras de datos propias (Cola Enlazada y Árbol Binario de Búsqueda) desde cero, sin utilizar las colecciones predefinidas de Java, demostrando la comprensión profunda de los conceptos de la materia. Además, utiliza una base de datos SQLite para la persistencia de la información, aplicando el patrón de diseño MVC (Modelo-Vista-Controlador) y principios de la metodología SOLID.

1.1 Justificación

Este proyecto surge de la necesidad de gestionar de manera ordenada y eficiente la atención de clientes, evitando la congestión y garantizando un orden estricto FIFO (First In, First Out) en la atención, mientras se mantiene un historial persistente de las operaciones.

1.2 Alcance

El sistema permite registrar y gestionar clientes y servicios, generar turnos automáticamente, atenderlos en orden de llegada, cancelar turnos pendientes, y consultar historiales y reportes resumidos por servicio, incluyendo filtrado por fechas.

2. OBJETIVOS

2.1 Objetivo General

Desarrollar una aplicación de consola en Java que gestione turnos de atención al cliente utilizando estructuras de datos propias y persistencia en base de datos relacional.

2.2 Objetivos Específicos

1. Implementar una Cola Enlazada genérica (ColaEnlazada<T>) para gestionar turnos pendientes respetando el orden FIFO.
2. Implementar un Árbol Binario de Búsqueda (BST) para almacenar y buscar clientes de manera eficiente por código.
3. Aplicar persistencia de datos utilizando SQLite y JDBC, con el patrón DAO e interfaces**.**
4. Validar todas las reglas de negocio (ej. evitar que un cliente tenga dos turnos pendientes simultáneos).
5. Documentar el proceso de desarrollo, las pruebas realizadas y la evidencia del trabajo colaborativo en Git.

3. MARCO TEÓRICO

3.1 Estructuras de Datos Implementadas

- Cola Enlazada (ColaEnlazada.java): Estructura lineal que sigue el principio FIFO. Los elementos se insertan por el final y se eliminan por el frente. Utiliza la clase Nodo<T> para las referencias. Operaciones: encolar, desencolar, consultarPrimero, cancelarPorNumero y recorrer.
- Árbol Binario de Búsqueda (BST.java): Estructura no lineal donde cada nodo tiene como máximo dos hijos. Los nodos del subárbol izquierdo son menores que la raíz, y los del derecho son mayores. Utiliza la clase NodoBST.
    - *Optimización:* La búsqueda se implementó con un enfoque iterativo (usando ciclos while) para reducir el consumo de memoria en el stack de llamadas, mientras que el recorrido inorden se mantiene recursivo para garantizar el orden natural de impresión de los clientes.

3.2 Patrón de Diseño: MVC y SOLID

- Modelo: Entidades Cliente, Servicio, Turno y ResumenServicio.
- Vista: MenuPrincipal.java (interacción con el usuario).
- Controlador/Negocio: ServicioNegocio.java y las clases DAO.
- Se aplica el Principio de Responsabilidad Única (SOLID): cada clase tiene un único propósito (ej. TurnoDAOImpl solo maneja la base de datos, no la lógica de la cola).

4. METODOLOGÍA Y DISTRIBUCIÓN DE RESPONSABILIDADES

El equipo se organizó siguiendo la distribución sugerida por el catedrático, asignando un módulo específico a cada integrante. Posteriormente, se realizó una fase de refactorización donde cada miembro optimizó su área asignada, dejando evidencia en el historial de Git.

Integrante	Módulo Asignado	Responsabilidades Principales	Refactorización y Aportes en Git
Rubí E. Marroquín C. (0905-24-11744)	**Clientes y BST**	- Implementación del BST y clase `Cliente.java`. - Arquitectura general MVC e integración de capas. - Clases `Inicializador.java` y `Main.java`.	- Optimización del método `buscar()` en BST (de recursivo a iterativo). - Diseño de interfaces DAO (`IClienteDAO`, etc.) para desacoplamiento.
Ericson J. Cristales R. (5190-24-10720)	**Turnos y Cola Enlazada**	- Implementación de `ColaEnlazada.java` y `Turno.java`. - Pruebas de funcionamiento (QA) y validación de casos de uso. - Documentación del sistema.	- Agregación de validación de datos nulos en `encolar()`. - Mejora en el manejo de excepciones al desencolar. - Refactorización de métodos `toString()` en el modelo.
Kevin A. Navas Arias (5990-22-11393)	**Servicios, Persistencia y Reportes**	- Desarrollo de `Servicio.java` y `ResumenServicio.java`. - Implementación de `SQLiteConnection.java` y los DAOs. - Generación de reportes y resúmenes por servicio.	- Optimización de los DAOs con `try-with-resources` para evitar fugas de memoria. - Refactorización del `MenuPrincipal.java` usando `switch` moderno y `String.format()`.

5. DESARROLLO DEL PROYECTO

5.1 Estructura del Repositorio

El proyecto en GitHub (rmarroquinc9/sistema-turnos-grupo9) sigue la siguiente estructura de paquetes:

```
GRUPO 9/
├── lib/
│   └── sqlite-jdbc-3.36.0.3.jar
├── sql/
│   └── crear_y_cargar.sql
├── src/main/java/com/turnos/
│   ├── Inicializador.java   # Crea tablas y carga datos iniciales
│   ├── Main.java            # Punto de entrada de la aplicación
│   ├── dao/                 # Acceso a datos (Interfaces e Implementaciones)
│   ├── estructuras/         # BST, ColaEnlazada, Nodo, NodoBST
│   ├── modelo/              # Cliente, Servicio, Turno, ResumenServicio
│   ├── negocio/             # ServicioNegocio.java (Reglas de negocio)
│   └── vista/               # MenuPrincipal.java
├── pom.xml                  # Configuración de Maven
├── uml.png                  # Diagrama de Clases
└── er.png                   # Diagrama Entidad-Relación
```

6. MANUAL DE USUARIO

6.1 Requisitos Previos

- Java Development Kit (JDK) 21 instalado.
- Sistema operativo Windows, Linux o macOS.

6.2 Instalación y Ejecución

Opción 1: Usando Maven (Recomendado)

New-Item -ItemType Directory -Force target\classes

javac -encoding UTF-8 -cp "lib\sqlite-jdbc-3.36.0.3.jar" -d target\classes (Get-ChildItem -Recurse -Filter *.java src\main\java).FullName

java -cp "target\classes;lib\sqlite-jdbc-3.36.0.3.jar" com.turnos.Inicializador

java -cp "target\classes;lib\sqlite-jdbc-3.36.0.3.jar" com.turnos.Main

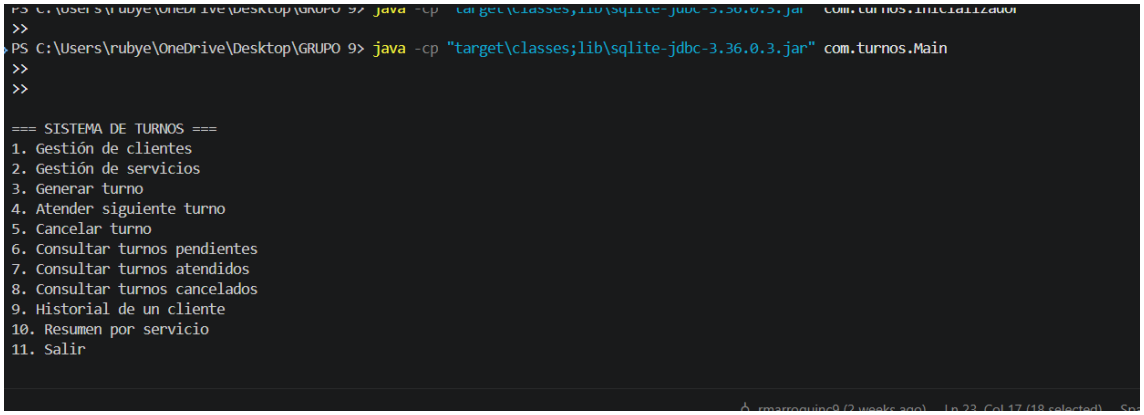
6.3 Flujo de Uso

1. Ejecutar Inicializador para crear la base de datos y cargar datos de prueba.
2. Ejecutar Main para abrir el menú principal.
3. Registrar/Consultar clientes y servicios (Opciones 1 y 2).
4. Generar un turno (Opción 3), atender (Opción 4) o cancelar (Opción 5).
5. Consultar reportes: pendientes, atendidos, cancelados, historial y resumen por servicio (Opciones 6 a 10).

7. EVIDENCIAS DE DESARROLLO Y REFACTORIZACIÓN

7.1 Capturas de Pantalla del Sistema

Menú principal



Búsqueda de Cliente en BST



Generación y Atención de Turnos

```
Problems 1  Output  Debug Console  Terminal  Ports

PS C:\Users\rubye\OneDrive\Desktop\GRUPO 9> java -cp "target\classes;lib\sqlite-jdbc-3.36.0.3.jar" com.turnos.Main
10. Resumen por servicio
11. Salir
Seleccione una opción: 3
Código del cliente: 1001
Código del servicio: 1
Error: El cliente ya tiene un turno pendiente
```

Validación (Turno Duplicado)

```
Problems 1  Output  Debug Console  Terminal  Ports

PS C:\Users\rubye\OneDrive\Desktop\GRUPO 9> java -cp "target\classes;lib\sqlite-jdbc-3.36.0.3.jar" com.turnos.Main
10. Resumen por servicio
11. Salir
Seleccione una opción: 3
Código del cliente: 1001
Código del servicio: 1
Error: El cliente ya tiene un turno pendiente
```

Reporte por Servicio

```
Seleccione una opción: 10

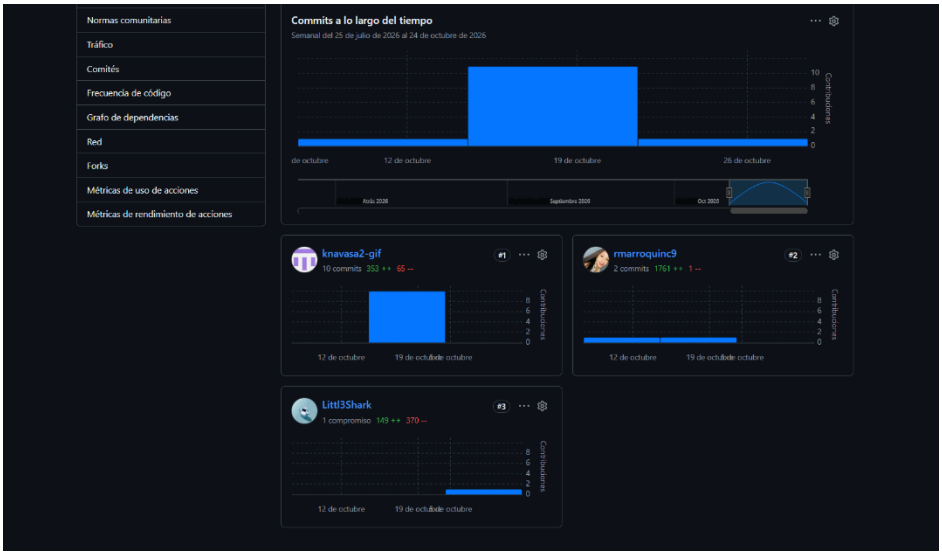
--- FILTROS DEL RESUMEN POR SERVICIO ---
Fecha inicial (AAAA-MM-DD, Enter para omitir):
Fecha final (AAAA-MM-DD, Enter para omitir):
Servicios:
- 1: Información
- 2: Entrega de documentos
- 3: Actualización de datos
Código de servicio (Enter para todos):
Estado (PENDIENTE/ATENDIDO/CANCELADO, Enter para todos):

--- RESUMEN POR SERVICIO ---
Servicio                        Pendientes   Atendidos  Cancelados    Total
-------------------------------------------------------------------------------
1: Información                           0           2           1        3
2: Entrega de documentos                 0           2           0        2
3: Actualización de datos                0           0           0        0
-------------------------------------------------------------------------------
TOTAL GENERAL                            0           4           1        5
Servicio con mayor demanda: Información (3 turnos)
```

8. EVIDENCIAS DE TRABAJO EN EQUIPO

8.1 Contribuciones en GitHub

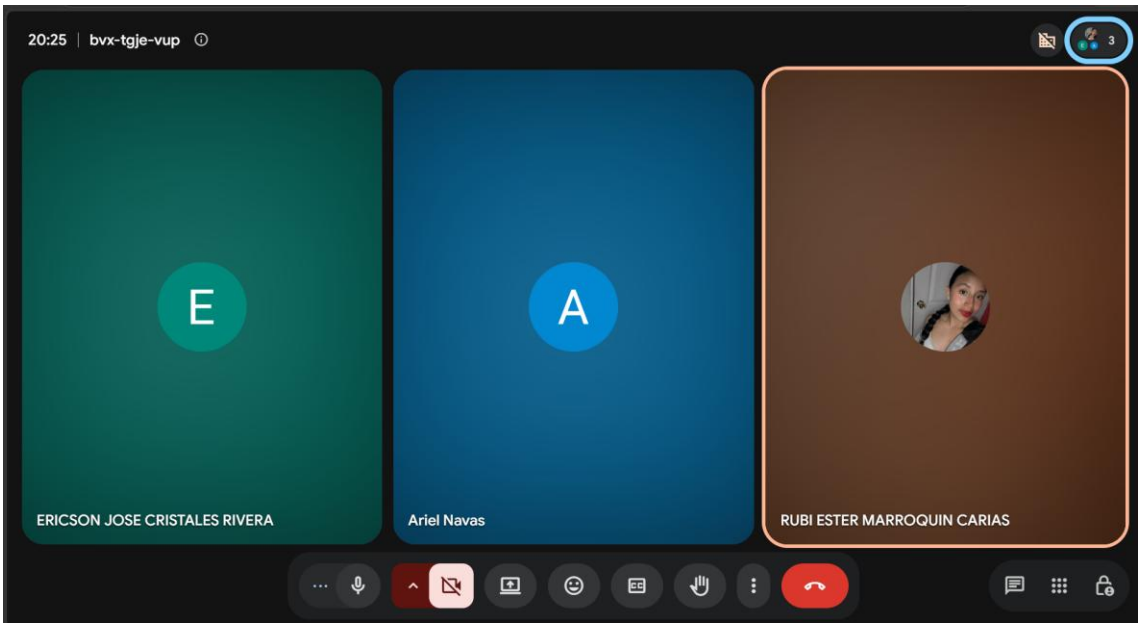
El repositorio cuenta con la participación activa y registrada de los tres integrantes, cumpliendo con el requisito de evidencia de aportes.



Gráfica de contribuciones donde se observan los commits y aportes de los 3 integrantes (rmarroquinc9, Littl3Shark, knavasa2-gif).

8.2 Sesiones de Trabajo

Evidencia de las sesiones de trabajo colaborativo para la integración y pruebas del sistema



9. CONCLUSIONES

1. **Dominio de Estructuras Propias:** Se logró implementar con éxito un BST y una Cola Enlazada desde cero, comprendiendo a fondo el manejo de nodos, punteros y la complejidad algorítmica.
2. **Persistencia Robusta:** La integración con SQLite mediante JDBC y el patrón DAO permitió que la información se mantuviera entre sesiones, reconstruyendo la cola de turnos al reiniciar el programa.
3. **Calidad de Código (Refactorización):** La fase final de refactorización grupal (uso de try-with-resources, validaciones nulas, búsqueda iterativa y switch modernos) mejoró significativamente la eficiencia, legibilidad y robustez del sistema.
4. **Trabajo Colaborativo Real:** El uso de Git y GitHub fue fundamental para el control de versiones, permitiendo una distribución clara del trabajo y dejando evidencia verificable del aporte individual de cada integrante.
5. **Aplicación de Patrones:** El uso del patrón MVC y el principio de Responsabilidad Única (SOLID) resultó en un código modular, fácil de mantener y de probar

10. RECOMENDACIONES

1. **Para Futuras Versiones:** Implementar una interfaz gráfica de usuario (GUI) con JavaFX o Swing, y agregar pruebas unitarias automatizadas con JUnit.
2. **Para la Base de Datos:** Agregar índices en las columnas de búsqueda frecuente (como codigoCliente) para optimizar el rendimiento en volúmenes de datos masivos.
3. **Para el Desarrollo:** Mantener la práctica de revisiones de código (Code Review) entre compañeros antes de hacer merge a la rama principal, tal como se inició en este proyecto.

11. BIBLIOGRAFÍA

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3ra ed.). MIT Press.
2. Oracle Corporation. (2023). *Java SE 21 Documentation*. Recuperado de https://docs.oracle.com/javase/
3. SQLite. (2023). *Official Documentation*. Recuperado de https://www.sqlite.org/docs.html
4. Martin, R. C. (2000). *Design Principles and Design Patterns*. Prentice Hal

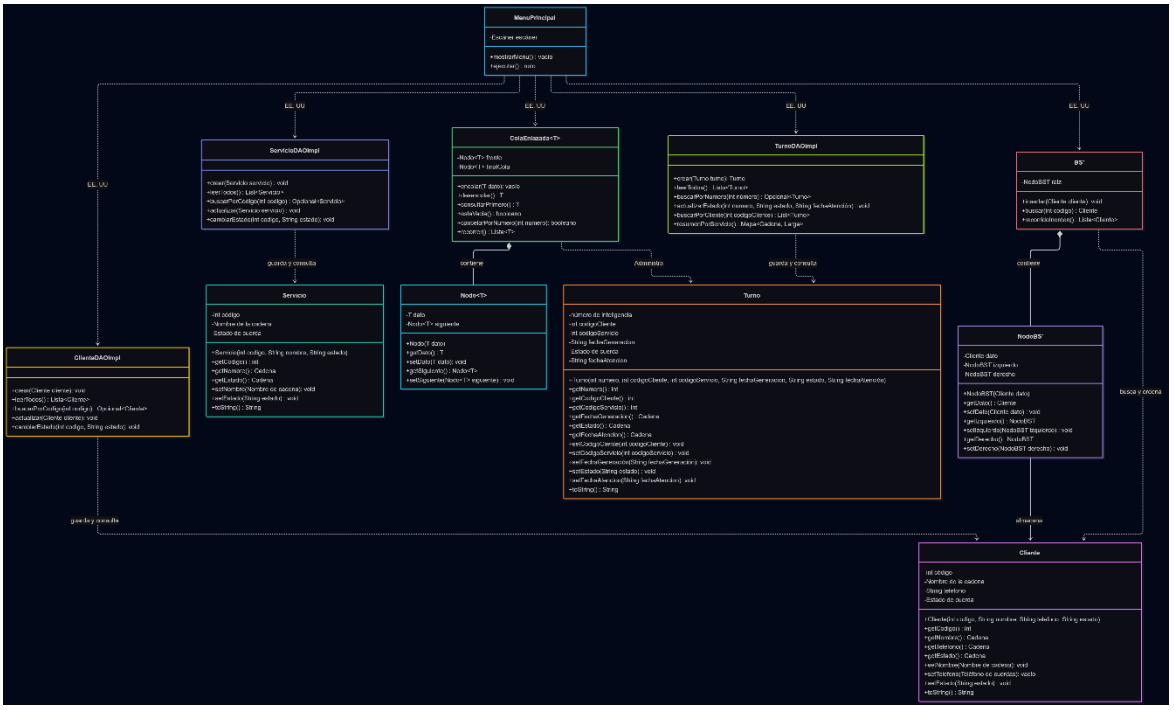
12. ANEXOS

Anexo A: Repositorio Oficial del Proyecto

El código fuente completo, historial de commits y diagramas están disponibles en:

URL: https://github.com/rmarroquinc9/sistema-turnos-grupo9

Anexo B: Diagrama de Clases (UML)



Anexo C: Diagrama Entidad-Relación (DER)

